

Mobile Development :

8 : Advanced State Management Cubit/BloC



Professor Imed Bouchrika

National School of Artificial Intelligence
imed.bouchrika@ensia.edu.dz

Outline :

- **Revision:**
 - *Design Patterns*
 - *GetX as a Simple State Management*
- **Bloc**
 - *Cubit*
 - *Provider*
 - *Consumer*
 - *States*
- **Integration with Storage Solutions**

Section 1

Revision...



Revision : Singleton Design Pattern



- **Singleton Pattern**
 - A technique to ensure only one instance object is created
- **What's a design pattern:**
 - Technique or a solution to a common problem in programming.

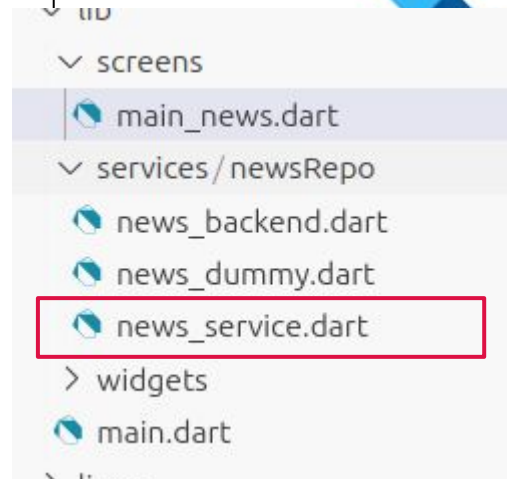
Revision : Singleton Design Pattern

```
import 'package:news_app/services/newsRepo/news_dummy.dart';

abstract class AbstractNewsRepo {
  Future<List<Map>> getNewsData();

  static AbstractNewsRepo? _newsInstance;

  static AbstractNewsRepo getInstance() {
    _newsInstance ??= NewsDummy();
    return _newsInstance!; // For backend data
  }
}
```



Revision : Factory Design Pattern



- **Factory Pattern**

- Same as the singleton, but does not ensure only one active instance is running.
- It is a creational design pattern providing an interface for creating objects with the possibility to :
 - Choose the concrete type from the abstraction.
 - Add further business logic when creating an object.
 - Remove totally the “new” and reduce the level of coupling (= **decoupling**)

Revision : Design Pattern : Provider



- **Provider**
 - The Provider Design Pattern is a structural design pattern that facilitates the **provision** of objects, services or dependencies to **other classes** or software components in a controlled, flexible, and reusable manner.

Revision

- Design Patterns :
 - Factory Pattern vs. Provider vs. Service Locator

	<i>Factory</i>	<i>Provider</i>	<i>Service Locator</i>
<i>Objective</i>	Focuses on creating objects.	Manage and supply dependencies to consumers.	Provide a central access point for service lookup.
<i>Coupling</i>	Tight coupling to factory	Loose coupling between provider and consumers.	Consumers depend on a central service locator.

Design Pattern : Subscribe-Publish

- **Subscribe-Publish :**

- It is a messaging design pattern where senders (publishers) and receivers (subscribers) communicate indirectly through a centralized broker.
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Design Pattern : Subscribe-Publish

Senders are totally decoupled from the receivers.

They are also decoupled from a temporal level (asynchronous communication)

- **Subscribe-Publish :**

- It is a messaging design pattern where senders (publishers) and receivers (subscribers) communicate indirectly through a centralized broker.
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Design Pattern : Subscribe-Publish

- **Subscribe-Publish :**

- It is a messaging design pattern where publishers (publishers) communicate indirectly with subscribers (subscribers) communicate indirectly
- Publishers emit messages about events or changes without needing to know who will receive them
- ⇒ while subscribers register interest in specific events and are notified by the broker when those events occur.

Students who are interested to receive news about AI for Health prepared by me, subscribe to the list called "ai-health" .. when i write a news related to this topic, i publish it to this list, it gets broadcast to all students..



Design Pattern : Observer

- **Observer**

- is a behavioral pattern where an object (**subject**), maintains a list of its observers (dependents)
- ⇒ **notifies** them of any state changes, typically by calling their update methods.
- The subject **must know** their dependents.



Revision

- **State Management :**

- Refers to the process of refreshing UI Widgets/Screens whenever the data bound for the widget state is modified.
- Single data change can impact multiple independent widgets. (Even on separate screens)
- The business logic for managing and updating data should be decoupled from the UI whilst state management libraries would take care of refreshing the UI whenever it is needed.

Revision

- **State Management :**

- What about using setState ?

- Pro:

- Easy to use
 - No dependencies

- But :

- Not suitable/maintainable/extensible when :
 - There are relations between Widgets
 - Want to share states across different widgets/screens

Revision

- **State Management using GetX:**

- Claiming to be the most popular library within the flutter pub.dev
- Set of libraries offering various functionalities including:
 - State Management
 - Routing and Navigation
 - ...
- Why GetX : Performance + Coding Productivity + Decoupling

Revision

- **State Management using GetX:**

- Claiming to be the most popular library within the flutter pub dev

- Set of

- S

- R

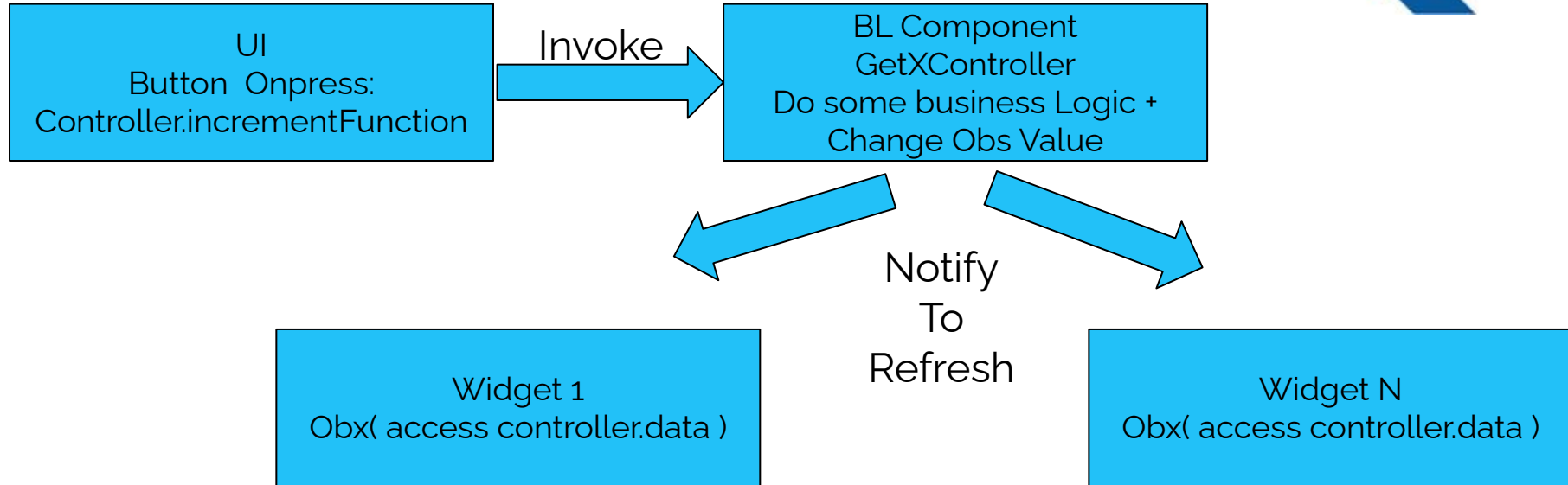
- ...

Remember to create Stateless Widget whilst using State Management Libraries, it will be creating implicitly the stateful widget.

⇒ Use Stateful Widget + Forget setState

- Why GetX : Performance + Coding Productivity + Decoupling

Revision : GetX



```

import 'package:flutter/material.dart';
import 'package:get/get.dart';
import 'controllers/increment.dart';
import 'widgets/myA.dart';
import 'widgets/myB.dart';
void main() {
  runApp(MainApp());
}
class MainApp extends StatelessWidget {
  MainApp({super.key});
  final IncrementController controller = Get.put(IncrementController());
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text("State Management")),
        body: Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              MyA(controller: controller),
              SizedBox(height: 30),
              MyB(controller: controller)
            ],
          ),
        ),
      );
  }
}

```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

```

import 'package:flutter/material.dart';
import 'package:get/get.dart';

import '../controllers/increment.dart';
import '../main.dart';

class MyA extends StatelessWidget {
  IncrementController controller;
  MyA({super.key, required this.controller});

  @override
  Widget build(BuildContext context) {
    return Row(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Obx(() =>
          Text("Widget A : Value is ${controller.increment}")),
        SizedBox(width: 20),
        ElevatedButton(
          onPressed: () {
            controller.doIncrement();
          },
          child: Text("Increment"))
      ],
    );
  }
}

```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

State Management in Flutter

```
import 'package:get/get.dart';

class IncrementController extends GetxController {
  final increment = 0.obs;

  @override
  void onInit() {
    super.onInit();
  }

  @override
  void onReady() {}

  @override
  void onClose() {}

  void doIncrement() {
    increment.value = increment.value + 1;
  }
}
```

State Management

DEBUG

Widget A : Value is 7

Increment

Widget B : value is 1

Revision

- **Async & Future**

```
Future<int> getInt() async{  
    return 1;  
}
```

Revision

- **Async & Future**

```
Stream<int> getInt() async*{  
    var i=0;  
  
    while(true){  
        yield i++;  
    }  
}
```

Section 2

Cubit/BloC for State Management



Business Logic, Data, UI

- We said :
 - Never put business logic code within the UI.
 - But still, you can call `setState` to refresh the UI based on the data accessible within the same file
 - What if we say :
 - Never put the business logic inside the same file as the UI :
 - Of course, you compute/generate data
 - How to `setState` to other widgets that has relation with this data ?

BloC

- **Definition**

- Bloc = **Business Logic Component**
 - is a powerful state management library in Flutter that helps separate business logic from the user interface.
- To use it inside **Flutter** :
 - `flutter pub add flutter_bloc`

BloC

- **BloC Libraries**

- **Cubit**

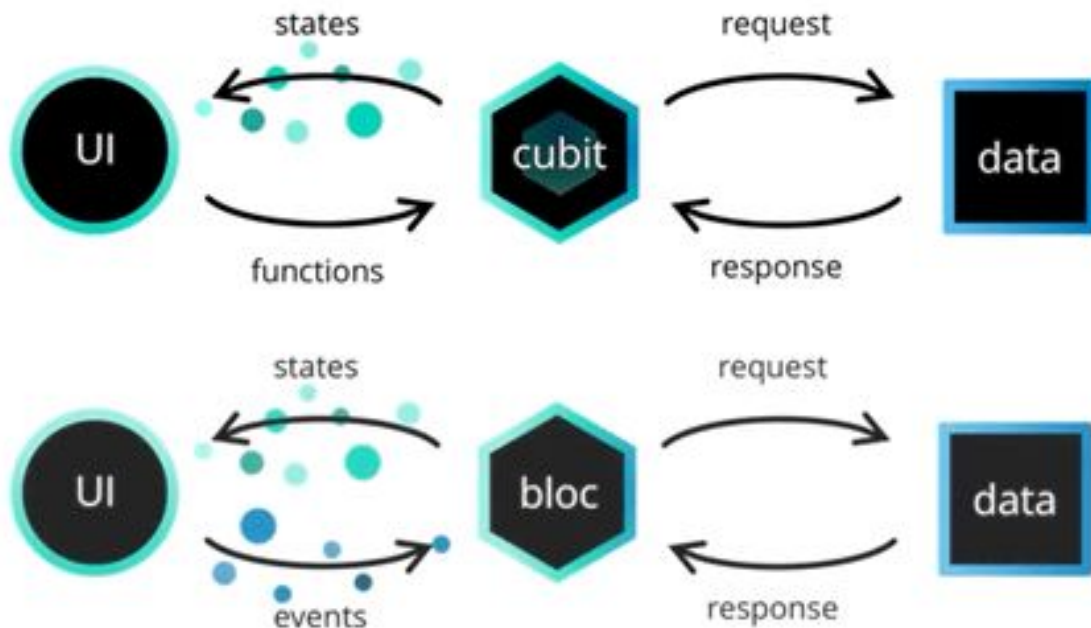
- A lightweight state management class for managing a single state.
 - Emits new states when triggered by methods.

- **Bloc:**

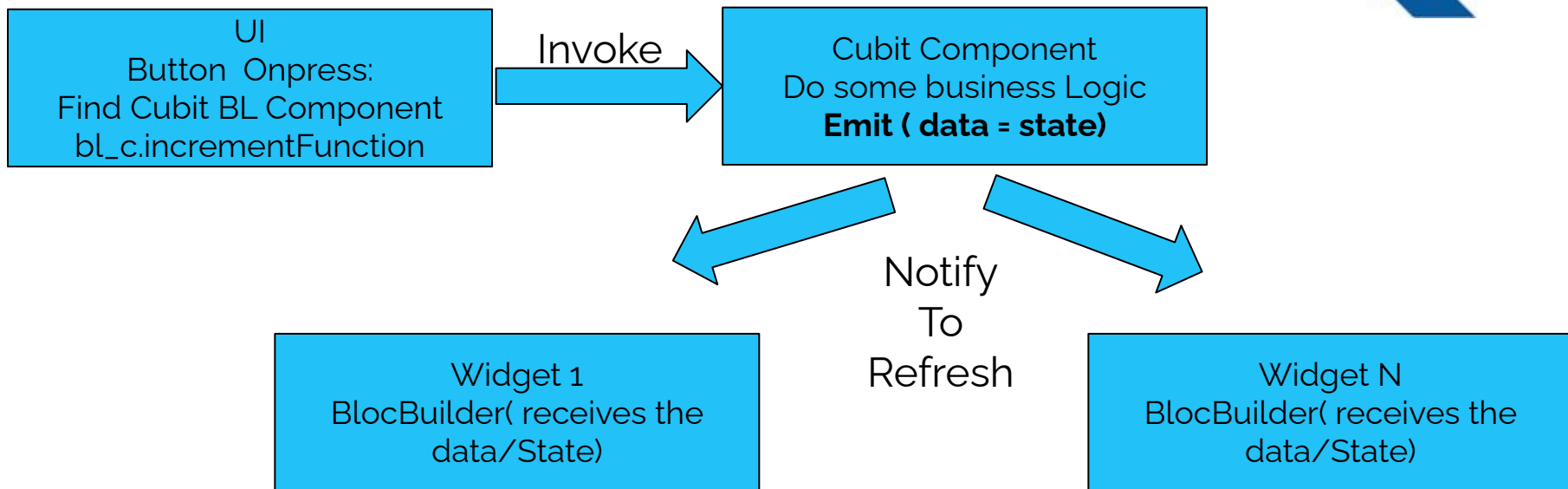
- Extends Cubit but introduces an event-based approach.
 - Reacts to incoming events and maps them to states.

BloC

- Flow of Actions



BloC/Cubit



BloC

- **Flow of Actions**

1. User Interaction: Begins with a user interaction

- *User clicks "Increment" button*

2. Event Dispatch (BloC) or Method Invocation (Cubit):

This interaction dispatches an event to the BLoC.

- *Within onPressed of the UI code, dispatch an event ..*

3. The business Logic Component processes the event/method invocation

BLoC

- **Flow of Actions**

4. State Generation & **Emit** : The BLoC yields and **emits** a new state with **data**
 - State can be a new primitive value, an object ...
5. All concerned widgets listening to the Bloc/Cubit , will be refreshed:
 - When there is an emit instruction.

BloC

- **Why to you must use BloC**

- **Clear Separation of Concerns:**

- Business logic is separate from UI, making code easier to maintain and test.

- **Reactiveness:**

- Widgets automatically rebuild when a new state is emitted.

- **Powerful State Management:**

- States are managed in a structured, **predictable way** with possibility to share them across widgets, screens...

- **Testability:**

- Business logic is encapsulated in Bloc and can be tested independently of UI.

BloC

- BloC Widgets

- ***BlocBuilder:***

- Like the FutureBuilder, it rebuilds a widget in response to state changes **continuously**

- ***BlocListener:***

- Reacts to state changes but does **NOT** rebuild widgets.

- ***BlocConsumer:***

- **Combines** BlocBuilder and BlocListener for both rebuilding and reacting to states.

- ***BlocProvider + MultiBlocProvider***

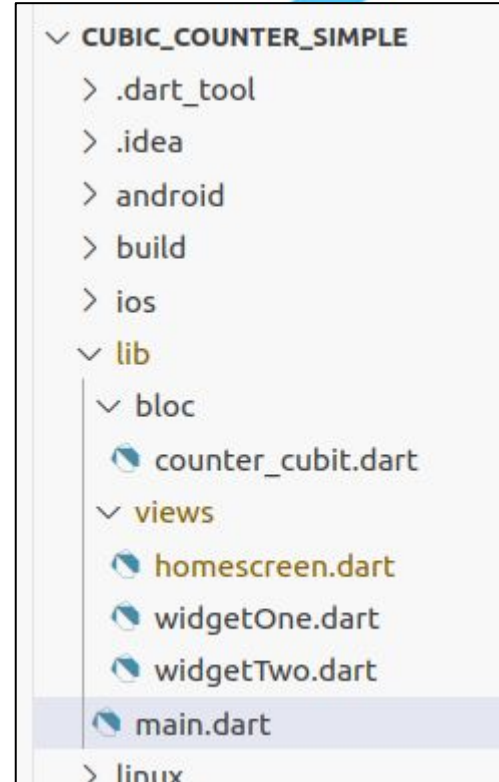
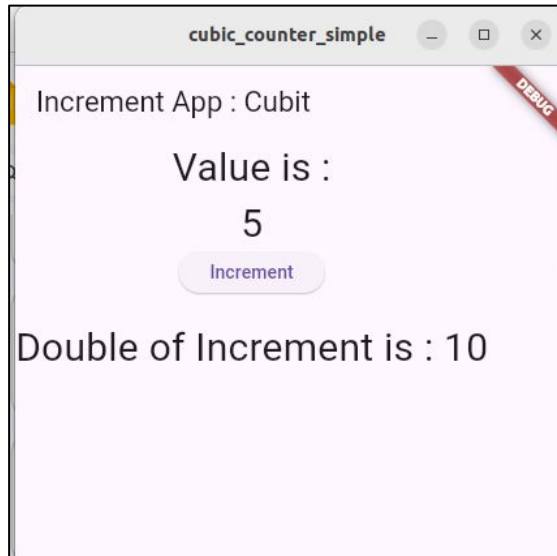
- Provides Bloc Objects to the widget tree



Simple Cubit /Int

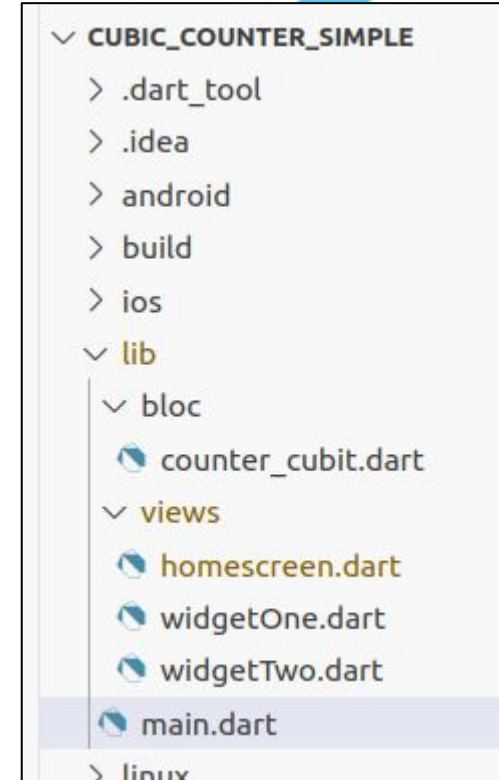
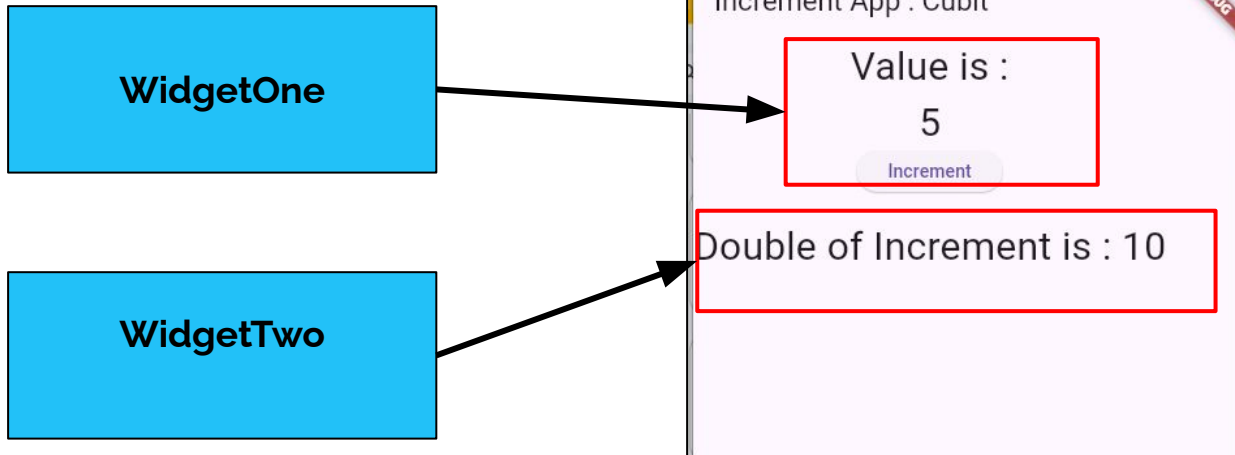
BloC

- Simple Incrementing App using BloC/Cubit



BloC

- Simple Incrementing App using BloC/Cubit



main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart' ;
import 'package:flutter/material.dart' ;
import 'package:flutter_bloc/flutter_bloc.dart' ;

import 'bloc/counter_cubit.dart' ;

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider (
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp (
        home: HomeScreen(),
      ));
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

counter_cubit.dart

✓ views

homescreen.dart

widgetOne.dart

widgetTwo.dart

main.dart

> linux

01

main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/counter_cubit.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

**Providers : Create an instance of X or Z
and provide them to other components
or classes where it can be injected with
all its resources and dependencies
easily.
“Loose Coupling”**

main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import 'bloc/counter_cubit.dart';

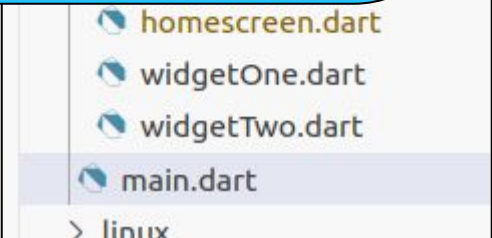
void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

Almost the same as :

**Get.put (some object) to be located /
provided later on ..**



homescreen.dart
widgetOne.dart
widgetTwo.dart
main.dart

> linux

Homescreen.dart

```
import 'package:cubic_counter_simple/views/widgetTwo.dart';
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../bloc/counter_cubit.dart';
import 'widgetOne.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Increment App : Cubit'),
      ),
      body: Column(
        children: [
          WidgetOne(),
          SizedBox(
            height: 20,
          ),
          WidgetTwo(),
        ],
      ),
    );
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

🔗 counter_cubit.dart

✓ views

🔗 homescreen.dart

🔗 widgetOne.dart

🔗 widgetTwo.dart

🔗 main.dart

> linux

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterCubit, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterCubit>().increment();
        },
        child: Text('Increment')),
    ]);
  }
}
```

**Find a CounterCubit Object .. (That will
be PROVISIONED by the BlocProvider
system**

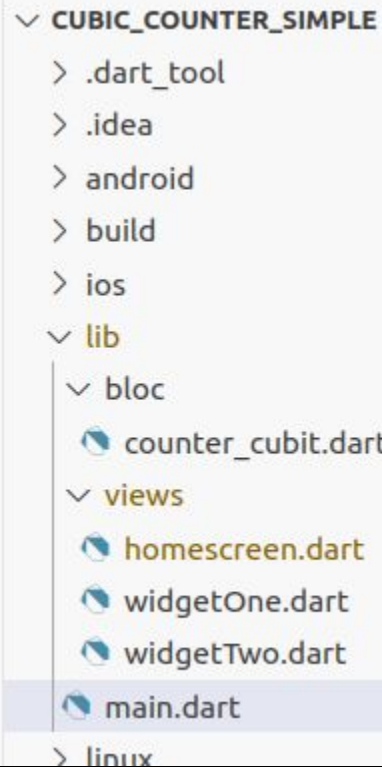
02

counter_cubit.dart

```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```



The image shows a file explorer sidebar from an IDE. At the top, there is a pink bar and a blue bar. The sidebar is titled 'CUBIC_COUNTER_SIMPLE'. It contains a list of folders and files: '.dart_tool', '.idea', 'android', 'build', 'ios', 'lib', 'views', and 'linux'. The 'lib' folder is expanded, showing 'bloc' (expanded) and 'views'. Under 'bloc', there is a file 'counter_cubit.dart'. Under 'views', there are files 'homescreen.dart', 'widgetOne.dart', and 'widgetTwo.dart'. The 'main.dart' file is highlighted at the bottom of the list.

- ✓ CUBIC_COUNTER_SIMPLE
 - > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ✓ lib
 - ✓ bloc
 - counter_cubit.dart
 - ✓ views
 - homescreen.dart
 - widgetOne.dart
 - widgetTwo.dart
 - > linux

03

counter_cubit.dart

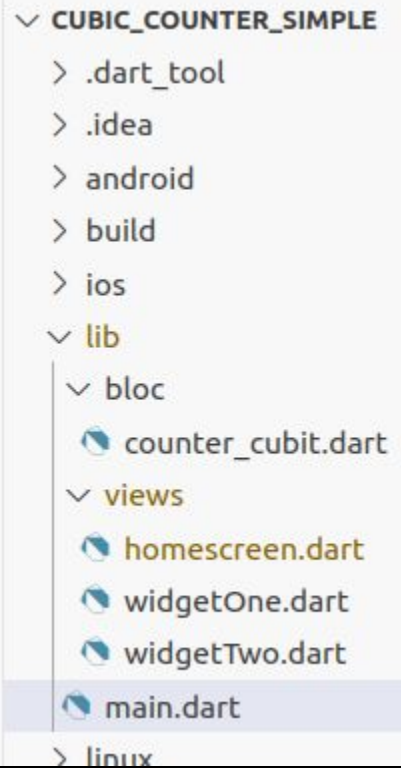
```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```

**state is a special instance variable
inherited from the Cubit**

**The state in this case is of type int
Initialized with an amount 0**



03

counter_cubit.dart

```
import 'package:bloc/bloc.dart';

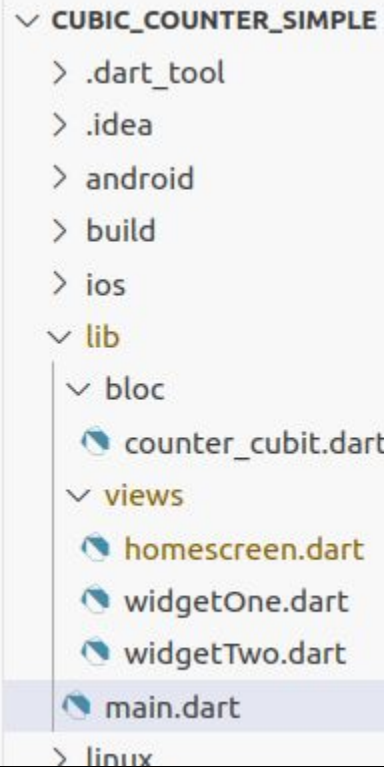
class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() { emit(state + 1) ; }
  void decrement() { emit(state - 1) ; }
}
```

Every time you **EMIT (SOME_DATA)**

⇒

SOME_DATA will be assigned to the instance
variable **state**
(**state = SOME_DATA**)

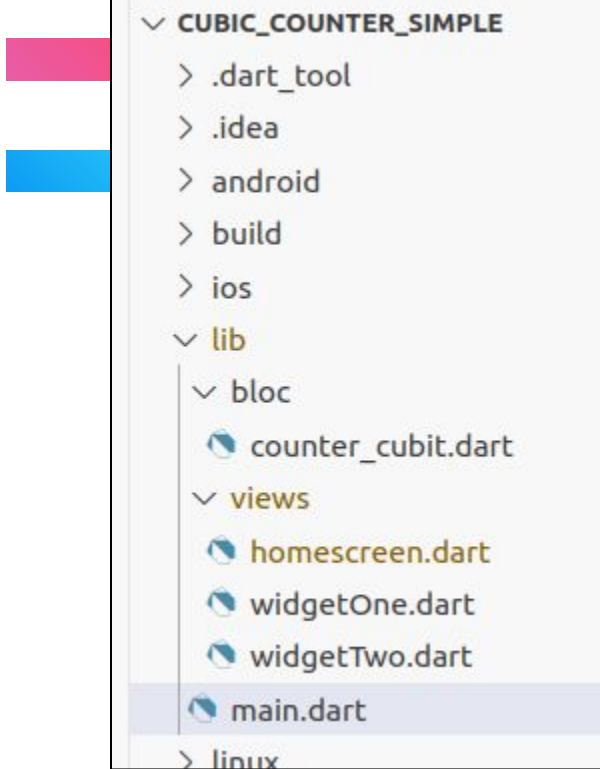


```
✓ CUBIC_COUNTER_SIMPLE
  > .dart_tool
  > .idea
  > android
  > build
  > ios
  ✓ lib
    ✓ bloc
      counter_cubit.dart
    ✓ views
      homescreen.dart
      widgetOne.dart
      widgetTwo.dart
  main.dart
  > linux
```

03

widgetOne.dart

```
class WidgetOne extends StatelessWidget {  
  const WidgetOne({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(children: [  
      Text(  
        'Value is :',  
        style: TextStyle(fontSize: 30),  
      ),  
      BlocBuilder<CounterCubit, int>(  
        builder: (context, count) {  
          return Text(  
            '$count',  
            style: TextStyle(fontSize: 30),  
          );  
        },  
      ),  
      ElevatedButton(  
        onPressed: () {  
          context.read<CounterCubit>().increment();  
        },  
        child: Text('Increment')),  
    ]);  
  }  
}
```



- ▼ CUBIC_COUNTER_SIMPLE
 - > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ▼ lib
 - ▼ bloc
 - counter_cubit.dart
 - ▼ views
 - homescreen.dart
 - widgetOne.dart
 - widgetTwo.dart
 - main.dart
 - > linux

04

widgetOne.dart

```
class WidgetOne extends StatelessWidget {  
  const WidgetOne({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return Column(children: [  
      Text(  
        'Value is :',  
        style: TextStyle(fontSize: 30),  
      ),  
      BlocBuilder<CounterCubit, int>(  
        builder: (context, count) {  
          return Text(  
            '$count',  
            style: TextStyle(fontSize: 30),  
          );  
        },  
      ),  
      ElevatedButton(  
        onPressed: () {  
          context.read<CounterCubit>().increment();  
        },  
        child: Text('Increment')),  
    ]);  
  }  
}
```

Like Obx in GetX

**BlocBuilder : will create a stateful
widgets containing the return of the
builder:**

**Importantly : will listen or observe the
instance PROVIDED of type
CounterCubit.**

The state is passed of type int*

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterCubit, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterCubit>().increment();
        },
        child: Text('Increment')),
    ]);
  }
}
```

Every time you **EMIT different state**,
BlocBuilder will be refreshed

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

> lib

art

t

> main.dart

> linux

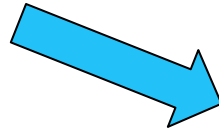
counter_cubit.dart

```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
}
```

builder: (context, count) : count is the actual state being passed.



emit (new state)

state is a special instance variable inherited from the Cubit.
It is like the “value” variable for GetX/Rx data objects.

```
@override
Widget build(BuildContext context) {
  return Column(children: [
    Text(
      'Value is :',
      style: TextStyle(fontSize: 30),
    ),
    BlocBuilder<CounterCubit, int>(
      builder: (context, count) {
        return Text(
          '$count',
          style: TextStyle(fontSize: 30),
        );
      },
    ),
    ElevatedButton(
      onPressed: () {
        context.read<CounterCubit>().increment();
      },
    ),
  ],
```

counter_cubit.dart

```
import 'package:bloc/bloc.dart';

class CounterCubit extends Cubit<int> {
  CounterCubit() : super(0);

  void increment() => emit(state + 1);
  void decrement() => emit(state - 1);
}
```

The state type here is primitive Int, Ideally, we can create our own Class CounterState instead with an instance variable : counter

To emit : ?

```
),
ElevatedButton(
  onPressed: () {
    context.read<CounterCubit>().increment(),
  },
)
```


Second widget

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
```

```
import '../bloc/counter/counter_cubit.dart';
```

```
class WidgetTwo extends StatelessWidget {
  const WidgetTwo({super.key});
```

```
  @override
```

```
  Widget build(BuildContext context) {
```

```
    return BlocBuilder<CounterCubit, int>(  
      builder: (context, count) {
```

```
        return Text(  
          'Double of Increment is : ${2 * count}',  
          style: TextStyle(fontSize: 30),  
        );  
      },  
    );  
  }
```

```
}
```

```
}
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Column(children: [
```

```
    Text(  
      'Value is :',  
      style: TextStyle(fontSize: 30),  
    ),  
    BlocBuilder<CounterCubit, int>(  
      builder: (context, count) {
```

```
        return Text(  
          '$count',  
          style: TextStyle(fontSize: 30),  
        );  
      },  
    ),  
    ElevatedButton(  
      onPressed: () {  
        context.read<CounterCubit>().increment();  
      },  
    ),  
  ],
```

```
  ),  
  ElevatedButton(  
    onPressed: () {  
      context.read<CounterCubit>().increment();  
    },  
  ),  
],
```

```
onPressed: () {
```

```
  context.read<CounterCubit>().increment();  
},
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ...

main.dart

```
import 'package:cubic_counter_simple/views/homescreen.dart' ;
import 'package:flutter/material.dart' ;
import 'package:flutter_bloc/flutter_bloc.dart' ;

import 'bloc/counter_cubit.dart' ;

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
      ],
      child: MaterialApp(
        home: HomeScreen(),
      ));
  }
}
```

✓ CUBIC_COUNTER_SIMPLE

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

🔗 counter_cubit.dart

✓ views

🔗 homescreen.dart

🔗 widgetOne.dart

🔗 widgetTwo.dart

🔗 main.dart

> linux

**Student's Question : What If i want to
have two separate CounterCubits ?**

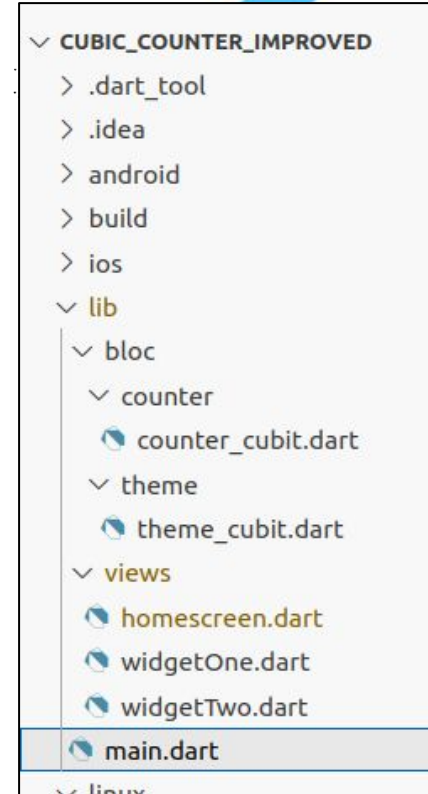
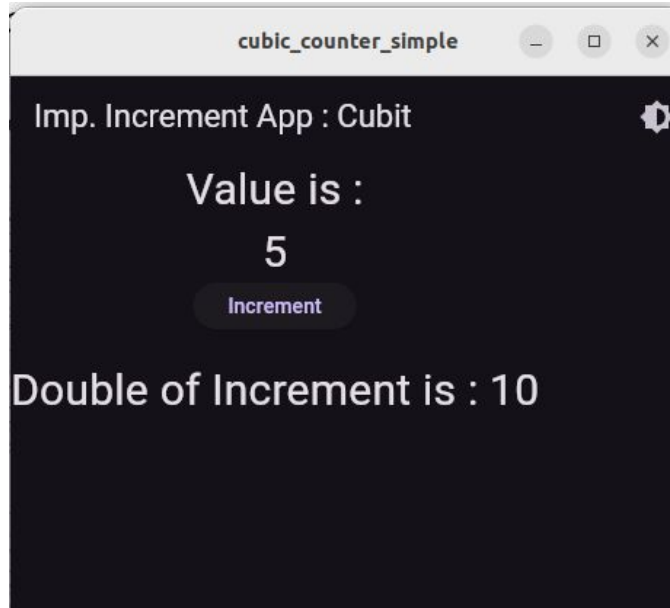
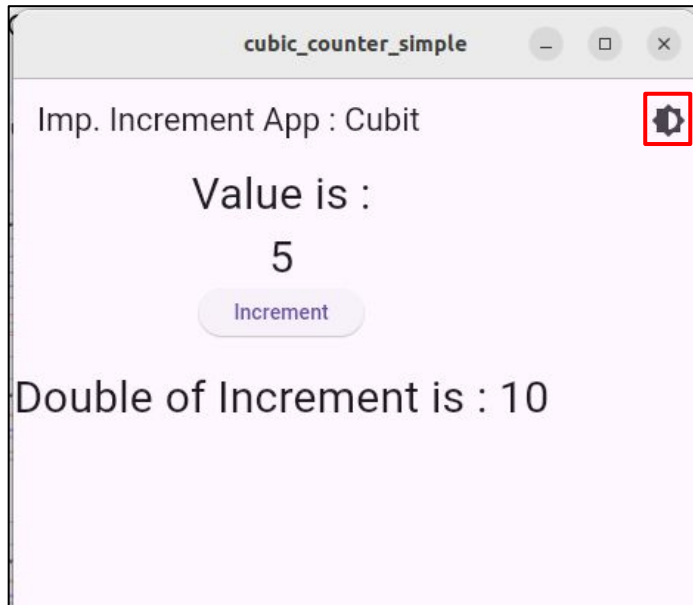
Asked by Abdelhak



Multiple Simple Cubits

BloC

- Improved Incrementing App using BloC/Cubit

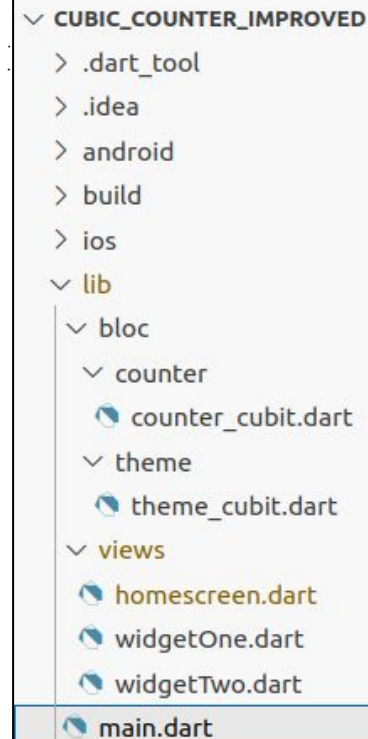


main.dart

```
void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => CounterCubit()),
        BlocProvider(create: (_) => ThemeCubit()),
      ],
      child: BlocBuilder<ThemeCubit, ThemeData>(builder: (context, theme)
{
    return MaterialApp(
      home: HomeScreen(),
      debugShowCheckedModeBanner: false,
      theme: theme,
    );
  }));
}
```

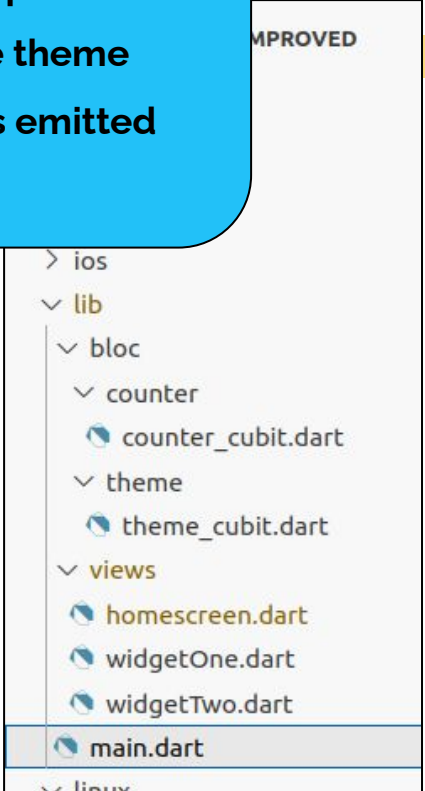


```
✓ CUBIC_COUNTER_IMPROVED
  > .dart_tool
  > .idea
  > android
  > build
  > ios
  ✓ lib
    ✓ bloc
    ✓ counter
      counter_cubit.dart
    ✓ theme
      theme_cubit.dart
    ✓ views
      homescreen.dart
      widgetOne.dart
      widgetTwo.dart
  main.dart
```

main.dart

```
void main() {  
  runApp(const MainApp());  
}  
class MainApp extends StatelessWidget {  
  const MainApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MultiBlocProvider(  
      providers: [  
        BlocProvider(create: (_) => CounterCubit()),  
        BlocProvider(create: (_) => ThemeCubit()),  
      ],  
      child: BlocBuilder<ThemeCubit, ThemeData>(builder: (context, theme) {  
        return MaterialApp(  
          home: HomeScreen(),  
          debugShowCheckedModeBanner: false,  
          theme: theme,  
        );  
      }));  
    }  
  }  
}
```

**The BlocBuilder is wrapping the
MaterialApp to update
automatically the theme
whenever a state is emitted**

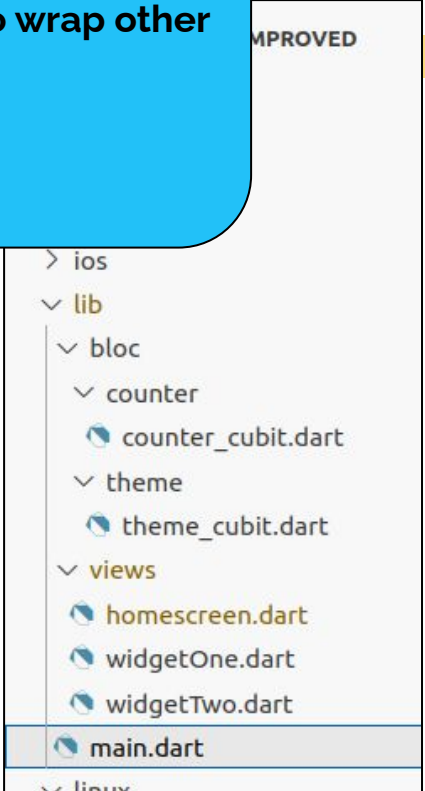


```
> ios  
✓ lib  
  ✓ bloc  
    ✓ counter  
      counter_cubit.dart  
    ✓ theme  
      theme_cubit.dart  
  ✓ views  
    homescreen.dart  
    widgetOne.dart  
    widgetTwo.dart  
  main.dart  
  < lib/ux
```

main.dart

```
void main() {  
  runApp(const MainApp());  
}  
  
class MainApp extends StatelessWidget {  
  const MainApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MultiBlocProvider(  
      providers: [  
        BlocProvider(create: (_) => CounterCubit()),  
        BlocProvider(create: (_) => ThemeCubit()),  
      ],  
      child: BlocBuilder<ThemeCubit, ThemeData>(builder: (context, theme) {  
        return MaterialApp(  
          home: HomeScreen(),  
          debugShowCheckedModeBanner: false,  
          theme: theme,  
        );  
      }));  
    }  
  }  
}
```

It is important to know when to
use the BlocBuilder to wrap other
widgets.



```
> ios  
✓ lib  
  ✓ bloc  
    ✓ counter  
      counter_cubit.dart  
    ✓ theme  
      theme_cubit.dart  
  ✓ views  
    homescreen.dart  
    widgetOne.dart  
    widgetTwo.dart  
  main.dart  
  < libux
```

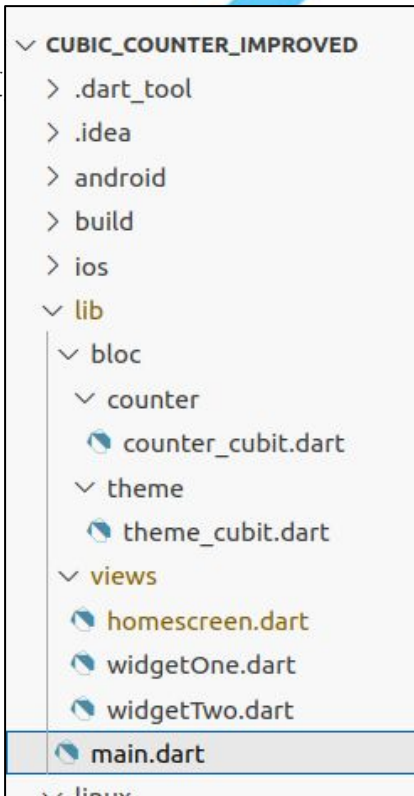
theme_cubit.dart

```
import 'package:bloc/bloc.dart';
import 'package:flutter/material.dart';

class ThemeCubit extends Cubit<ThemeData> {
  ThemeCubit() : super(ThemeData.light());

  void toggleTheme() {
    if (state.brightness == Brightness.light)
      emit(ThemeData.dark());
    else
      emit(ThemeData.light());
  }
}
```

The state/data here is a
ThemeData object instead of an
Integer



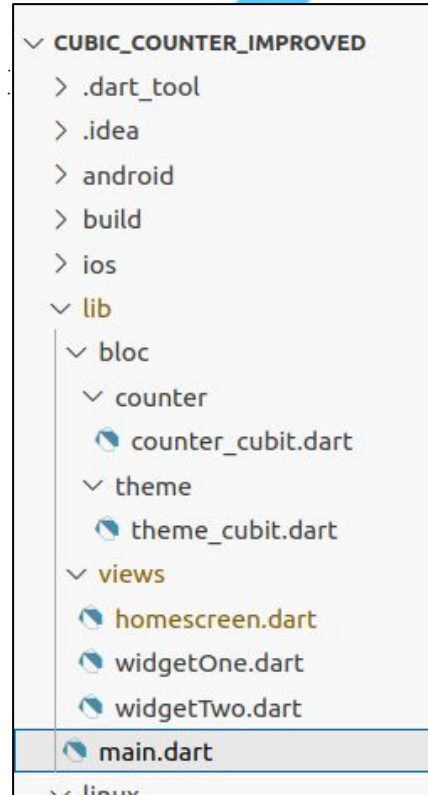

```

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Imp. Increment App : Cubit'),
        actions: [
          IconButton(
            icon: const Icon(Icons.brightness 6),
            onPressed: () => context.read<ThemeCubit>().toggleTheme(),
          ),
        ],
      ),
      body: Column(
        children: [
          WidgetOne(),
          SizedBox(
            height: 20,
          ),
          WidgetTwo(),
        ],
      ),
    );
  }
}

```

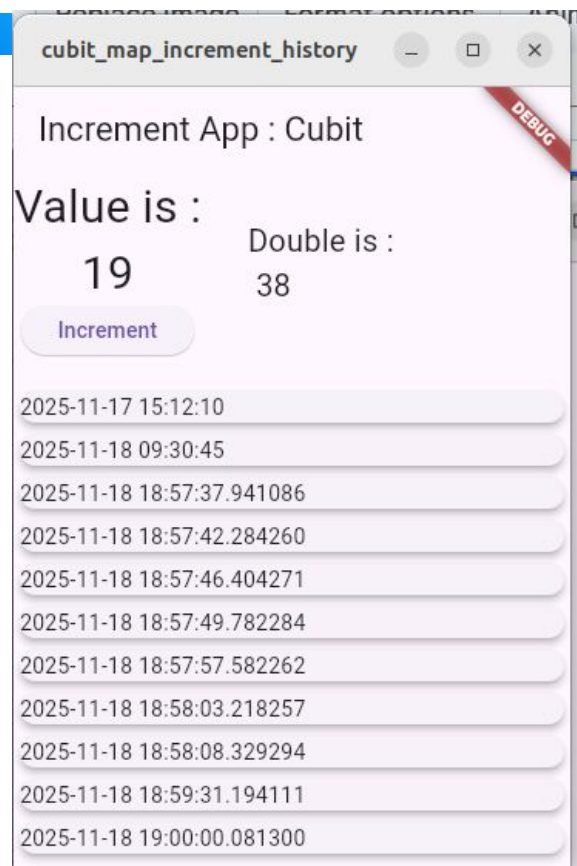
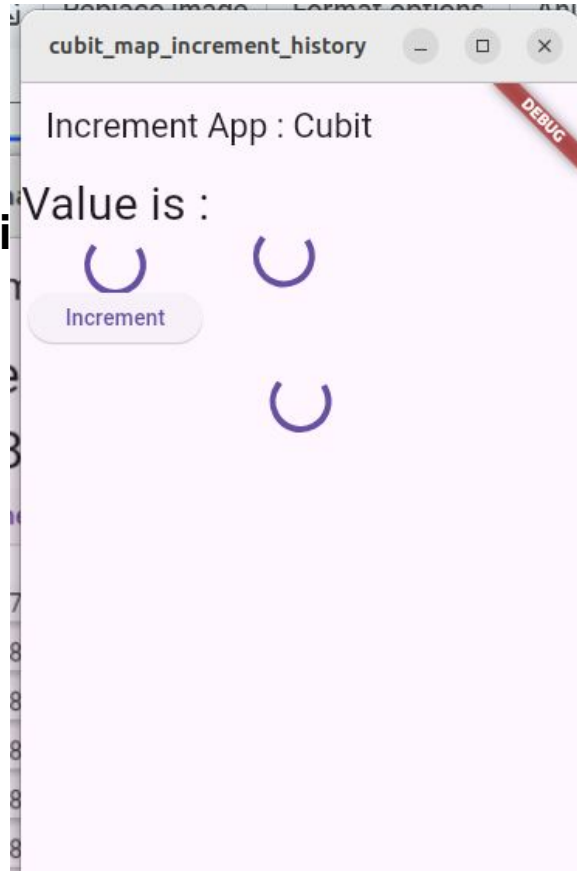
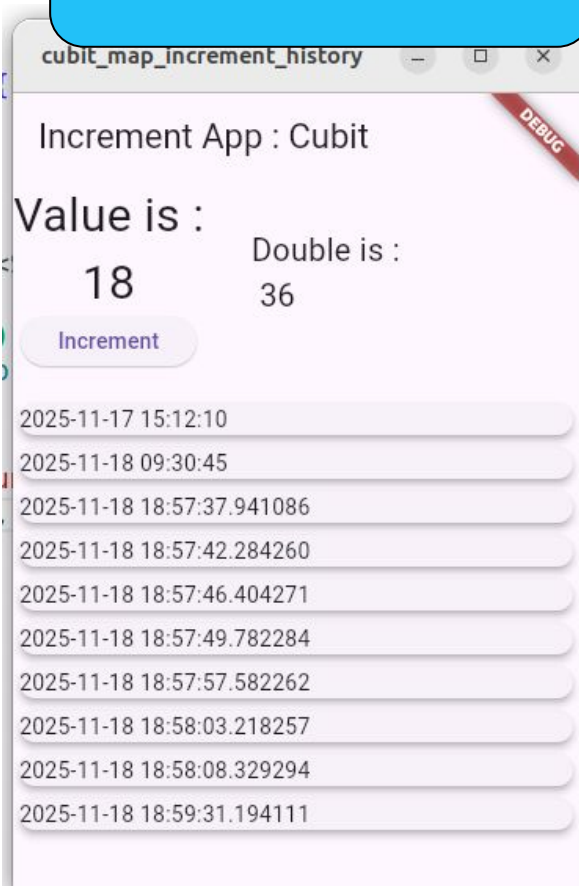
**Lookup an instance of type
ThemeCubit (Provider Pattern)**





Cubit States With Map

After clicking increment, it
takes some time to load...



main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

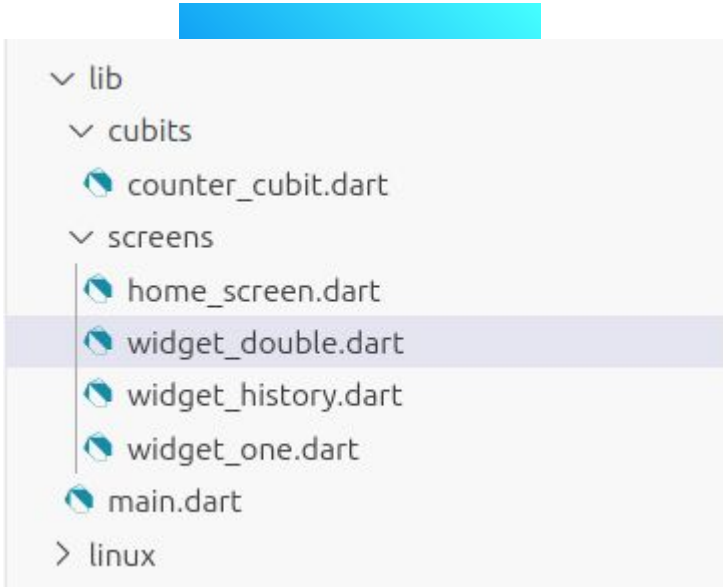
Future<bool> init_my_app() async {
  return true;
}

void main() async {
  await init_my_app();

  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [BlocProvider(create: (_) => CounterCubit())],
      child: MaterialApp(home: HomeScreen()),
    );
  }
}
```



- ▼ lib
 - ▼ cubits
 - counter_cubit.dart
 - ▼ screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart
 - > linux

home_screen.dart

```
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Increment App : Cubit')),
      body: Column(
        children: [
          Row(children: [WidgetOne(), SizedBox(width: 30), WidgetTwo()]),
          SizedBox(height: 20),
          WidgetHistory(),
        ],
      ),
    );
  }
}
```



- lib
 - cubits
 - counter_cubit.dart
 - screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
- linux

widget_one.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});
  @override
  Widget build(BuildContext context) {
    return Column(
      children: [
        Text('Value is :', style: TextStyle(fontSize: 30)),
        BlocBuilder<CounterCubit, Map<String, dynamic>>(
          builder: (context, state) {
            if (state['status'] == 'loading') {
              return CircularProgressIndicator();
            }
            if (state['status'] == 'error') {
              return Text('There is an error');
            }
            return Text('${state['count']}', style: TextStyle(fontSize: 30));
          },
        ),
        ElevatedButton(
          onPressed: () {
            context.read<CounterCubit>().increment();
          },
          child: Text('Increment'),
        ),
      ],
    );
  }
}
```

- lib
 - cubits
 - counter_cubit.dart
 - screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart
- linux

widget_one.dart

```
class CounterCubit extends Cubit<Map<String, dynamic>> {
  CounterCubit() : super({'status': 'loading', 'count': 0, 'history': []}) {
    loadData();
  }
  void loadData() {
    // Simulate loading data
    Map<String, dynamic> data = {
      'status': 'done',
      'count': 10,
      'history': ['2025-11-17 15:12:10', '2025-11-18 09:30:45'],
    };
    emit(data);
  }

  Future<bool> increment() async {
    emit({'status': 'loading', 'count': state['count'], 'history': state['history'],});
    await Future.delayed(Duration(seconds: 3));
    Map<String, dynamic> newState = {
      'status': 'done', 'count': state['count'], 'history': List<String>.from(state['history']),
    };
    newState['count'] += 1;
    newState['history'].add(DateTime.now().toString());
    emit(newState);
    return true;
  }
}
```

- lib
 - cubits
 - counter_cubit.dart
 - screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart
- > linux

widget_double.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';

import '../cubits/counter_cubit.dart';

class WidgetTwo extends StatelessWidget {
  const WidgetTwo({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<CounterCubit, Map<String, dynamic>>(
      builder: (context, state) {
        if (state['status'] == 'loading') {
          return CircularProgressIndicator();
        }

        return Text(
          'Double is :\n ${2 * state['count']}',
          style: TextStyle(fontSize: 20),
        );
      },
    );
  }
}
```

- ✓ lib
 - ✓ cubits
 - counter_cubit.dart
 - ✓ screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart
- > linux

widget_history.dart

```
class WidgetHistory extends StatelessWidget {
  const WidgetHistory({super.key});

  @override
  Widget build(BuildContext context) {
    return BlocBuilder<CounterCubit, Map<String, dynamic>>(
      builder: (context, state) {
        if (state['status'] == 'loading') {
          return CircularProgressIndicator();
        }
        return Expanded(
          child: ListView.builder(
            itemCount: state['history'].length,
            itemBuilder: (context, index) {
              return Card(
                elevation: 4,
                child: Text('${state['history'][index]}'),
              );
            },
          ),
        );
      },
    );
  }
}
```

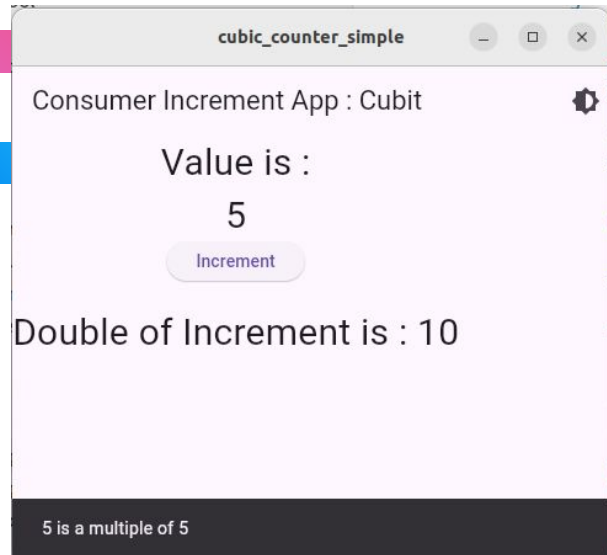
- lib
 - cubits
 - counter_cubit.dart
 - screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart
- linux



UI-Related Business Logic

BloC

- **Increment using BloC with BlocConsumer..**
 - Listening and doing some extra UI-Related business logic
 - When the counter % 5 == 0
 - Show snackbar message
 - When Counter == 13
 - Take user to a new DecrementScreen
 - In the DecrementScreen
 - If counter is even, show a snackbar message



Old widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});

  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocBuilder<CounterCubit, int>(
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
        onPressed: () {
          context.read<CounterCubit>().increment();
        },
        child: Text('Increment')),
    ]);
  }
}
```

✓ CUBIC_COUNTER_CONSUMER

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc

✓ counter

🔗 counter_cubit.dart

✓ theme

🔗 theme_cubit.dart

✓ views

🔗 decrement_screen.dart

🔗 homescreen.dart

🔗 widgetOne.dart

🔗 widgetTwo.dart

🔗 main.dart

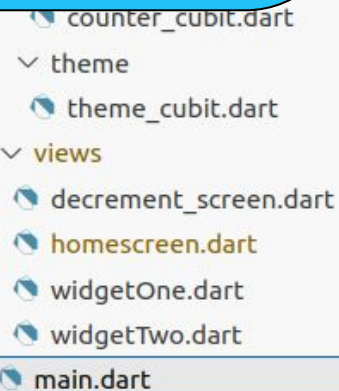
widgetOne.dart

```
class WidgetOne extends StatelessWidget {  
  const WidgetOne({super.key});  
  @override  
  Widget build(BuildContext context) {  
    return Column(children: [  
      Text(  
        'Value is :',  
        style: TextStyle(fontSize: 30),  
      ),  
      BlocConsumer<CounterCubit, int>(  
        listener: (context, count) {  
          if (count % 5 == 0) {  
            ScaffoldMessenger.of(context).showSnackBar(  
              SnackBar(  
                content: Text('$count is a multiple of 5'),  
                duration: Duration(milliseconds: 5000),  
              ),  
            );  
          }  
          if (count == 13) {  
            Navigator.push(  
              context,  
              MaterialPageRoute(builder: (context) => DecrementScreen()),  
            );  
          }  
        },  
        builder: (context, count) {  
          return Text(  
            '$count',  
            style: TextStyle(fontSize: 30),  
          );  
        },  
      ),  
      ElevatedButton(  

```

After doing business logic inside the Bloc and Emitting a state, we can listen to do further “side-effect” UI-related business logic.

You cannot do UI-related business logic inside the Bloc.



- counter_cubit.dart
- theme
- theme_cubit.dart
- views
 - decrement_screen.dart
 - homescreen.dart
 - widgetOne.dart
 - widgetTwo.dart
 - main.dart

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocConsumer<CounterCubit, int>(
        listener: (context, count) {
          if (count % 5 == 0) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(
                content: Text('$count is a multiple of 5'),
                duration: Duration(milliseconds: 500),
              ),
            );
          }
          if (count == 13) {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => DecrementScreen()),
            );
          }
        },
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
```

BlocConsumer will consume events ..(Producer-Consumer Pattern)

- ▼ counter
 - 🔍 counter_cubit.dart
- ▼ theme
 - 🔍 theme_cubit.dart
- ▼ views
 - 🔍 decrement_screen.dart
 - 🔍 homescreen.dart
 - 🔍 widgetOne.dart
 - 🔍 widgetTwo.dart
- 🔍 main.dart

widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocConsumer<CounterCubit, int>(
        listener: (context, count) {
          if (count % 5 == 0) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(
                content: Text('$count is a multiple of 5'),
                duration: Duration(milliseconds: 500),
              ),
            );
          }
          if (count == 13) {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => DecrementScreen()),
            );
          }
        },
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),
          );
        },
      ),
      ElevatedButton(
```

**When we go to the new Screen ?
shall we pass the increment
number ?**

▼ CUBIC_COUNTER_CONSUMER

- > .dart_tool
- > .idea
- > android
- > build

- homescreen.dart
- widgetOne.dart
- widgetTwo.dart
- main.dart

```

class DecrementScreen extends StatelessWidget {
  const DecrementScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Decrement Screen'),
        actions: [
          IconButton(
            icon: const Icon(Icons.brightness_6),
            onPressed: () => context.read<ThemeCubit>().toggleTheme(),
          ),
        ],
      ),
      body: Column(
        children: [
          Text('Value is :'),
          BlocConsumer<CounterCubit, int>(
            listener: (context, count) {
              if (count % 2 == 0) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(
                    content: Text('$count is even ? '),
                    duration: Duration(milliseconds: 300),
                  ),
                );
              }
            },
            builder: (context, count) {
              return Text(
                '$count',
              );
            },
          ),
          ElevatedButton(
            onPressed: () {

```

DRY ?

**When we decrement and reach 5
? will it show the SnackBar of
listener of WidgetOne ?**

▼ CUBIC_COUNTER_CONSUMER

- > .dart_tool
- > .idea

- decrement_screen.dart
- homescreen.dart
- widgetOne.dart
- widgetTwo.dart
- main.dart


```

class DecrementScreen extends StatelessWidget {
  const DecrementScreen({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('Decrement Screen'),
        actions: [
          IconButton(
            icon: const Icon(Icons.brightness_6),
            onPressed: () => context.read<ThemeCubit>().toggleTheme(),
          ),
        ],
      ),
      body: Column(
        children: [
          Text('Value is :'),
          BlocConsumer<CounterCubit, int>(
            listener: (context, count) {
              if (count % 2 == 0) {
                ScaffoldMessenger.of(context).showSnackBar(
                  SnackBar(
                    content: Text('$count is even ? '),
                    duration: Duration(milliseconds: 300),
                  ),
                );
              }
            },
            builder: (context, count) {
              return Text(
                '$count',
              );
            },
          ),
          ElevatedButton(
            onPressed: () {

```

**When we decrement and reach 5
? will it show the SnackBar of
listener of WidgetOne ?**

▼ CUBIC_COUNTER_CONSUMER

> .dart_tool

> .idea

decrement_screen.dart

homescreen.dart

widgetOne.dart

widgetTwo.dart

main.dart

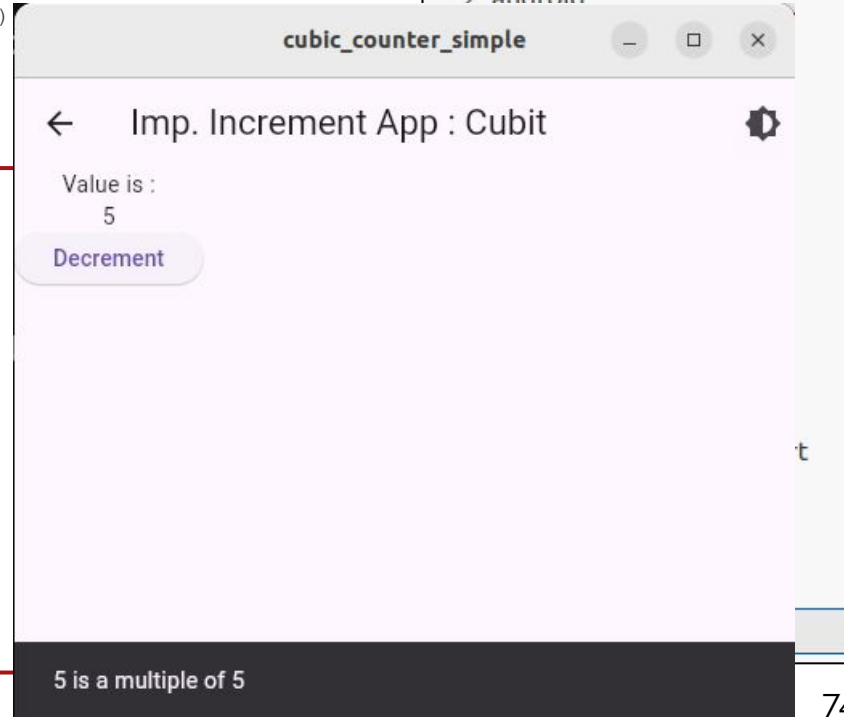
Because WidgetOne is still in still in the stack and its resources are still Active.

How to stop showing the messages of multiple of 5 from the child Screen ?

```
listener: (context, count) {  
  if (count % 2 == 0) {  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('$count is even ? '),  
        duration: Duration(milliseconds: 300),  
      ),  
    );  
  }  
},  
builder: (context, count) {  
  return Text(  
    '$count',  
  );  
},  
),
```

```
ElevatedButton(  
  onPressed: () {
```

```
t>().toggleTheme()
```



widgetOne.dart

```
class WidgetOne extends StatelessWidget {
  const WidgetOne({super.key});
  @override
  Widget build(BuildContext context) {
    return Column(children: [
      Text(
        'Value is :',
        style: TextStyle(fontSize: 30),
      ),
      BlocConsumer<CounterCubit, int>(
        listenWhen: (previous, current) {
          if (ModalRoute.of(context)?.isCurrent ?? false) return true;
          return false;
        },
        listener: (context, count) {
          if (count % 5 == 0) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(
                content: Text('$count is a multiple of 5'),
                duration: Duration(milliseconds: 5000),
              ),
            );
          }
          if (count == 13) {
            Navigator.push(
              context,
              MaterialPageRoute(builder: (context) => DecrementScreen()),
            );
          }
        },
        builder: (context, count) {
          return Text(
            '$count',
            style: TextStyle(fontSize: 30),

```

Use listenWhen :

▼ CUBIC_COUNTER_CONSUMER

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc

screen.dart

screen.dart

screen.dart

homescreen.dart

widgetOne.dart

widgetTwo.dart

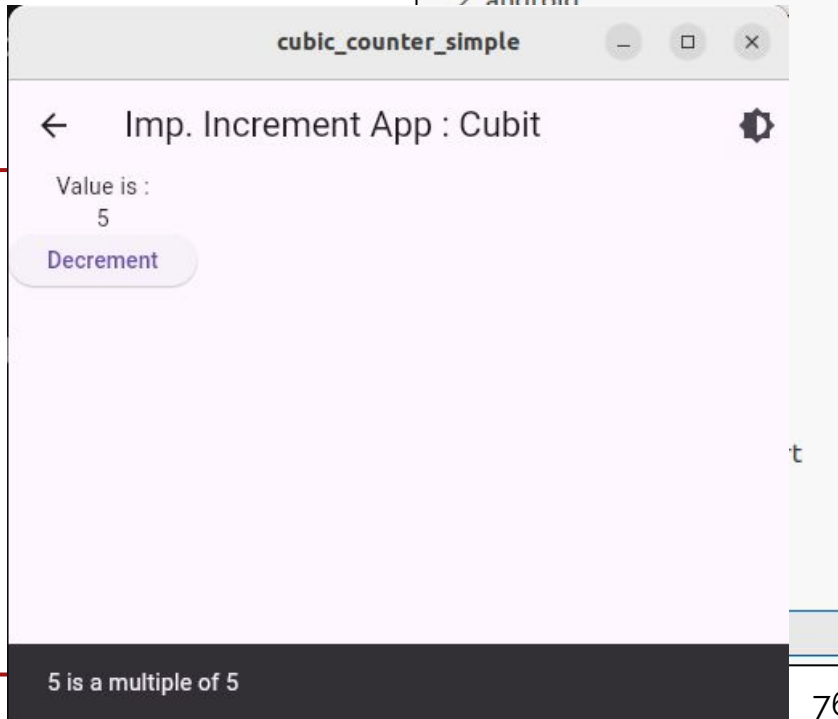
main.dart

If we go back to the ParentScreen ?
do we need to await and refresh the
screen to get the new increment
value ?

```
listener: (context, count) {  
  if (count % 2 == 0) {  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('$count is even ? '),  
        duration: Duration(milliseconds: 300),  
      ),  
    );  
  }  
},  
builder: (context, count) {  
  return Text(  
    '$count',  
  );  
},  
),
```

```
ElevatedButton(  
  onPressed: () {
```

```
t>().toggleTheme()
```



BloC

- Login Screen

- Exactly

- U
 - L
 - If
 - O

**Will the BlocBuilder refresh the
screen everytime we call Emit ?**

screen

id ...



Cubit States As a Class

BloC

- **Login Screen using BloC with states...**

- Exactly as we did with the FutureBuilder
 - Users puts their credentials and press Login
 - Loading circular will show for 3 seconds
 - If the username == 123, clear the stack of screens + take them to a new screen where you will be showing their username
 - Otherwise, show the login screen again with a snackBar message invalid ...

BloC

simple_login_states

Login Screen..

Username

Password

Login

simple_login_states

Login Screen..

simple_login_states

Login Screen..

Username 1111

Password

Login

Invalid username of password...

simple_login_states

Logged In

Welcome 123

Logged successfully...

main.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:simple_login_states/views/login.dart';

import 'bloc/login/login_cubit.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [
        BlocProvider(create: (_) => LoginCubit()),
      ],
      child: MaterialApp(
        home: LoginScreen(),
        debugShowCheckedModeBanner: false,
      ));
  }
}
```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc/login

login_cubit.dart

login_state.dart

▼ views

login.dart

welcome.dart

main.dart

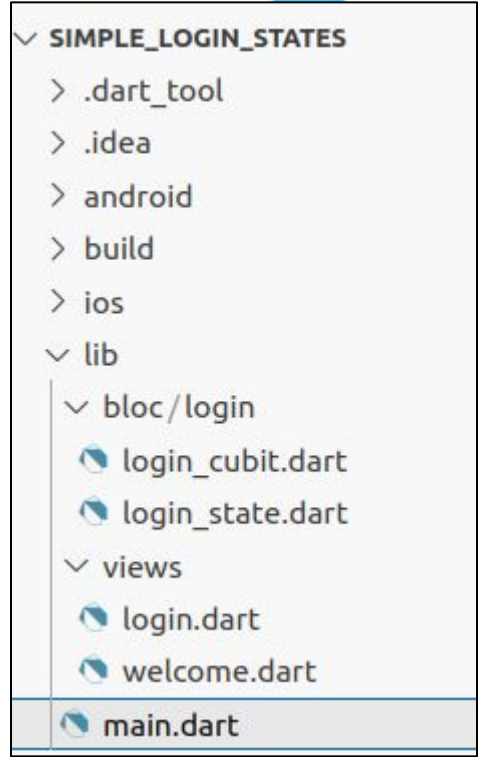
login_cubit.dart

```
import 'package:bloc/bloc.dart';
import 'package:simple_login_states/bloc/login/login_state.dart';

class LoginCubit extends Cubit<LoginState> {
  LoginCubit() : super(LoginInitial());

  Future<bool> actionProcessLogin(String username, String password) async {
    emit(LoginLoading());
    await Future.delayed(const Duration(seconds: 5)); // Simulate delay

    try {
      if (username == '123')
        emit(LoginSuccess(username)); // Emit success state
      else
        emit(LoginError('Invalid username or password...'));
    } catch (e) {
      emit(LoginError("An error occurred...")); // Emit error state
    }
    return true; // event driven architecture, has no meaning here :(
  }
}
```



▼ SIMPLE_LOGIN_STATES

- > .dart_tool
- > .idea
- > android
- > build
- > ios
- ▼ lib
 - ▼ bloc/login
 - login_cubit.dart
 - login_state.dart
 - ▼ views
 - login.dart
 - welcome.dart

main.dart

login_cubit.dart

```
import 'package:bloc/bloc.dart';
import 'package:simple_login_states/bloc/login/login_state.dart';

class LoginCubit extends Cubit<LoginState> {
  LoginCubit() : super(LoginInitial());

  Future<bool> actionProcessLogin(String username, String password) async {
    emit(LoginLoading());
    await Future.delayed(const Duration(seconds: 5)); // Simulate delay

    try {
      if (username == '123')
        emit(LoginSuccess(username)); // Emit success state
      else
        emit(LoginError('Invalid username or password...'));
    } catch (e) {
      emit(LoginError("An error occurred...")); // Emit error state
    }
    return true; // event driven architecture, has no meaning here :(
  }
}
```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

**This is a type of State that we
are creating**

login_state.dart

```
import 'package:equatable/equatable.dart';

abstract class LoginState extends Equatable {
  @override
  List<Object> get props => [];
}

class LoginInitial extends LoginState {
  @override
  List<Object> get props => ['loading'];
}

class LoginLoading extends LoginState {}

class LoginSuccess extends LoginState {
  final String username;
  @override
  List<Object> get props => ['logged'];

  LoginSuccess(this.username);
}

class LoginError extends LoginState {
  final String message;
  @override
  List<Object> get props => ['error'];
  LoginError(this.message);
}
```

**How to know that the current
state is different from the
newly emitted state to refresh**

?

login_state.dart

```
import 'package:equatable/equatable.dart';
```

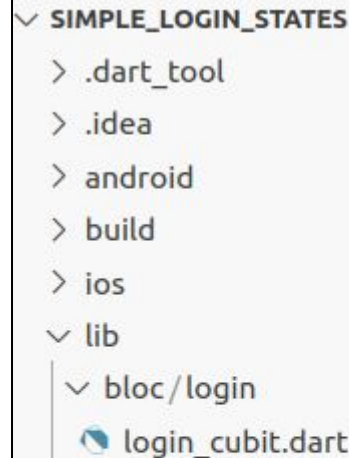
```
abstract class LoginState extends Equatable {  
  @override  
  List<Object> get props => [];  
}
```

```
class LoginInitial extends LoginState {  
  @override  
  List<Object> get props => ['loading'];  
}
```

```
class LoginLoading extends LoginState {}
```

```
class LoginSuccess extends LoginState {  
  final String username;  
  @override  
  List<Object> get props => ['logged'];  
  
  LoginSuccess(this.username);  
}
```

```
class LoginError extends LoginState {  
  final String message;  
  @override  
  List<Object> get props => ['error'];  
  LoginError(this.message);  
}
```



```
✓ SIMPLE_LOGIN_STATES  
  > .dart_tool  
  > .idea  
  > android  
  > build  
  > ios  
  ✓ lib  
    ✓ bloc/login  
      login_cubit.dart
```

flutter pub add equatable

views/login.dart

```
class LoginScreen extends StatelessWidget {  
  final TextEditingController usernameController = TextEditingController();  
  final TextEditingController passwordController = TextEditingController();  
  
  Widget getLoginUI(BuildContext context) {  
    return Column(  
      mainAxisAlignment: MainAxisAlignment.center,  
      children: [  
        // Username Field  
        TextField(  
          controller: usernameController,  
          decoration: InputDecoration(  
            labelText: 'Username',  
            prefixIcon: const Icon(Icons.person),  
            border: OutlineInputBorder(  
              borderRadius: BorderRadius.circular(8.0),  
            ),  
          ),  
        ),  
        const SizedBox(height: 16.0),  
        // Password Field  
        TextField(  
          controller: passwordController,  
          obscureText: true,  
          decoration: InputDecoration(  
            labelText: 'Password',  
            prefixIcon: const Icon(Icons.lock),  
            border: OutlineInputBorder(  
              borderRadius: BorderRadius.circular(8.0),  
            ),  
          ),  
        ),  
        const SizedBox(height: 24.0),  
        // Login Button  
      ],  
    );  
  }  
}
```

▼ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

▼ lib

▼ bloc/login

login_cubit.dart

login_state.dart

▼ views

login.dart

welcome.dart

main.dart

views/login.dart

```
class LoginScreen extends StatelessWidget {  
  final TextEditingController usernameController = TextEditingController();  
  final TextEditingController passwordController = TextEditingController();  
  
  const SizedBox(height: 24.0),  
  // Login Button  
  ElevatedButton(  
    onPressed: () {  
      String username = usernameController.text;  
      String password = passwordController.text;  
      context.read<LoginCubit>().actionProcessLogin(username, password);  
    },  
    style: ElevatedButton.styleFrom(  
      padding:  
        const EdgeInsets.symmetric(horizontal: 32.0, vertical: 12.0),  
    ),  
    child: const Text(  
      'Login',  
      style: TextStyle(fontSize: 18.0),  
    ),  
  ),  
],  
);  
}
```

✓ SIMPLE_LOGIN_STATES

> .dart_tool

> .idea

> android

> build

> ios

✓ lib

✓ bloc/login

login_cubit.dart

login_state.dart

✓ views

login.dart

welcome.dart

main.dart

views/login.dart

```
class LoginScreen extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login Screen..')),
      body: Center(
        child: BlocConsumer<LoginCubit, LoginState>(
          listener: (context, state) {
            if (state is LoginError) {
              // Show error snackbar
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text(state.message),
                  duration: Duration(seconds: 3),
                ),
              );
            }
            if (state is LoginSuccess) {
              Navigator.pushAndRemoveUntil(
                context,
                MaterialPageRoute(builder: (context) => WelcomeScreen()),
                (route) =>
                  false, // This condition removes all routes from the stack
              );

              // Show error snackbar
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text('Logged successfully...'),
                  duration: Duration(seconds: 3),
                ),
              );
            }
          },
        ),
      ),
    );
  }
},
```

▼ SIMPLE_LOGIN_STATES

- > .dart_tool
 - > .idea
 - > android
 - > build
 - > ios
 - ▼ lib
 - ▼ bloc/login
 - login_cubit.dart
 - login_state.dart
 - ▼ views
 - login.dart
 - welcome.dart
- main.dart

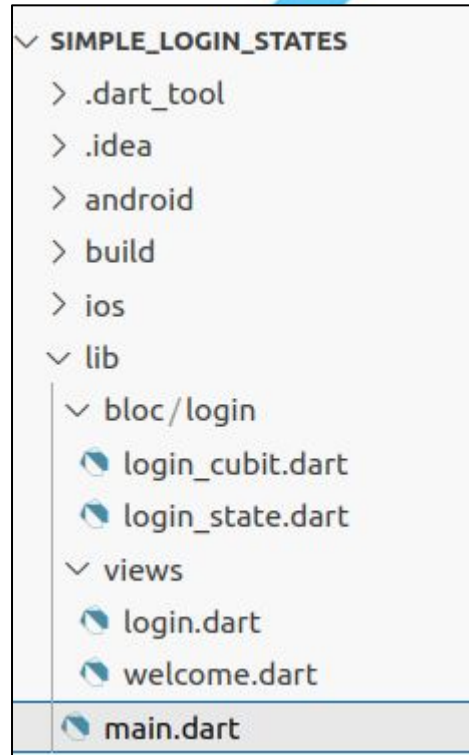
welcome.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_bloc/flutter_bloc.dart';
import 'package:simple_login_states/bloc/login/login_cubit.dart';

import '../bloc/login/login_state.dart';

class WelcomeScreen extends StatelessWidget {
  const WelcomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Logged In')),
      body: BlocBuilder<LoginCubit, LoginState>(builder: (context, state) {
        if (state is LoginSuccess)
          return Text(
            'Welcome ${state.username}',
            style: TextStyle(fontSize: 30),
          );
        else
          return Text(
            'Error,,,You should not be here...',
            style: TextStyle(fontSize: 30),
          );
      }));
  }
}
```

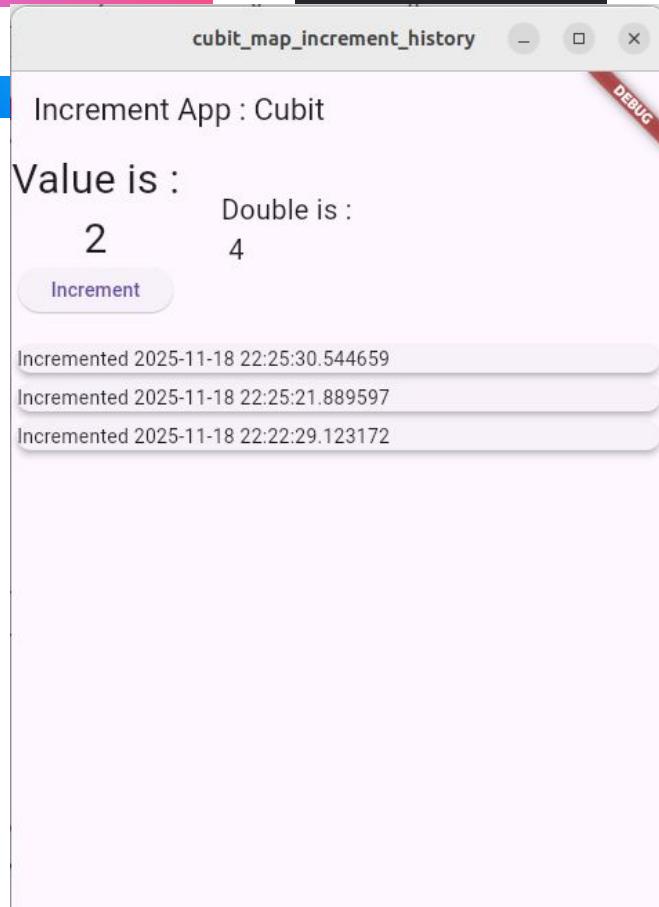




Cubit With Storage

BloC

- **Welcome Screen for Logged users :**
 - **Show the increment WidgetOne**
 - **Add History Widget**
 - **History should be saved to a database**



main.dart

```
Future<bool> init my app() async {
  final historyRepo = HistoryRepositoryBase.getInstance();
  return true;
}

void main() async {
  if (Platform.isLinux || Platform.isWindows) {
    sqfliteFfiInit();
    databaseFactory = databaseFactoryFfi;
  }

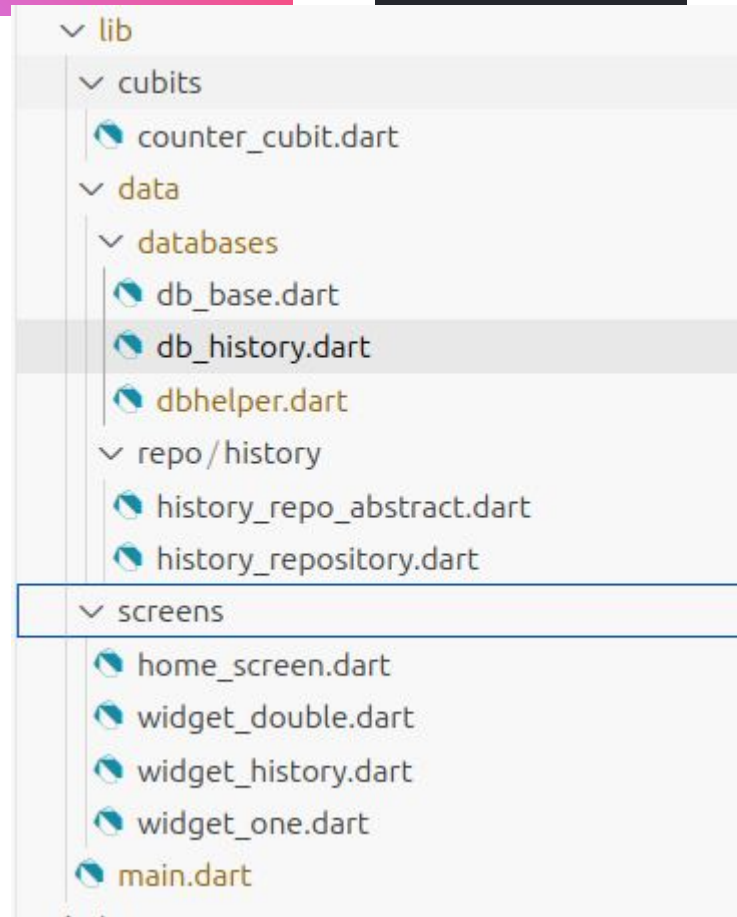
  WidgetsFlutterBinding.ensureInitialized();

  await init_my_app();

  runApp(MainApp());
}

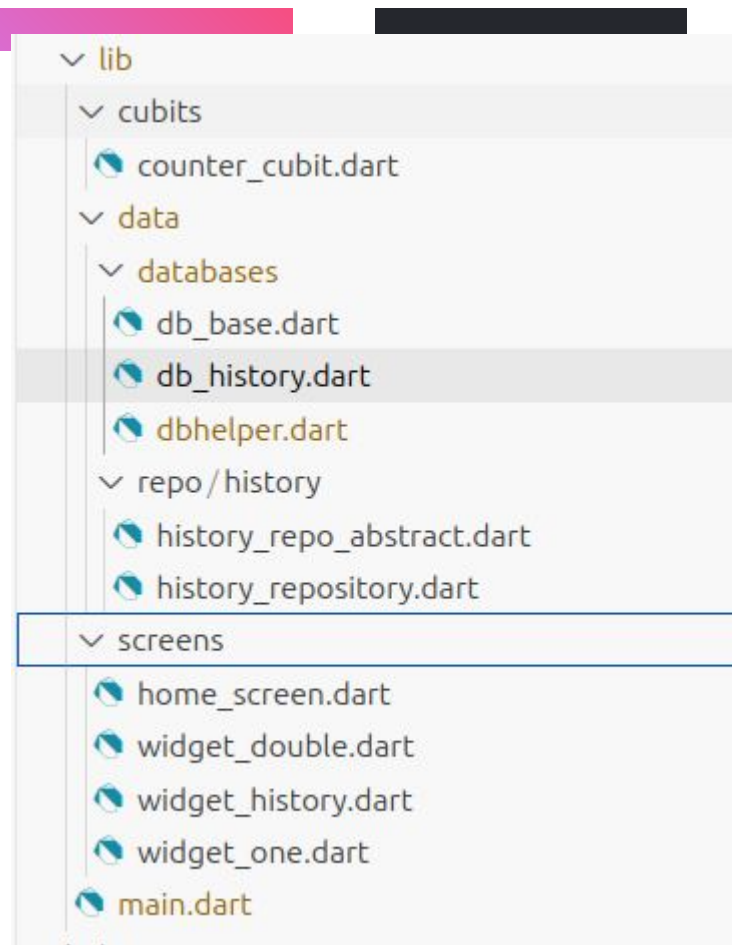
class MainApp extends StatelessWidget {
  MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MultiBlocProvider(
      providers: [BlocProvider(create: (_) => CounterCubit())],
      child: MaterialApp(home: HomeScreen()),
    );
  }
}
```



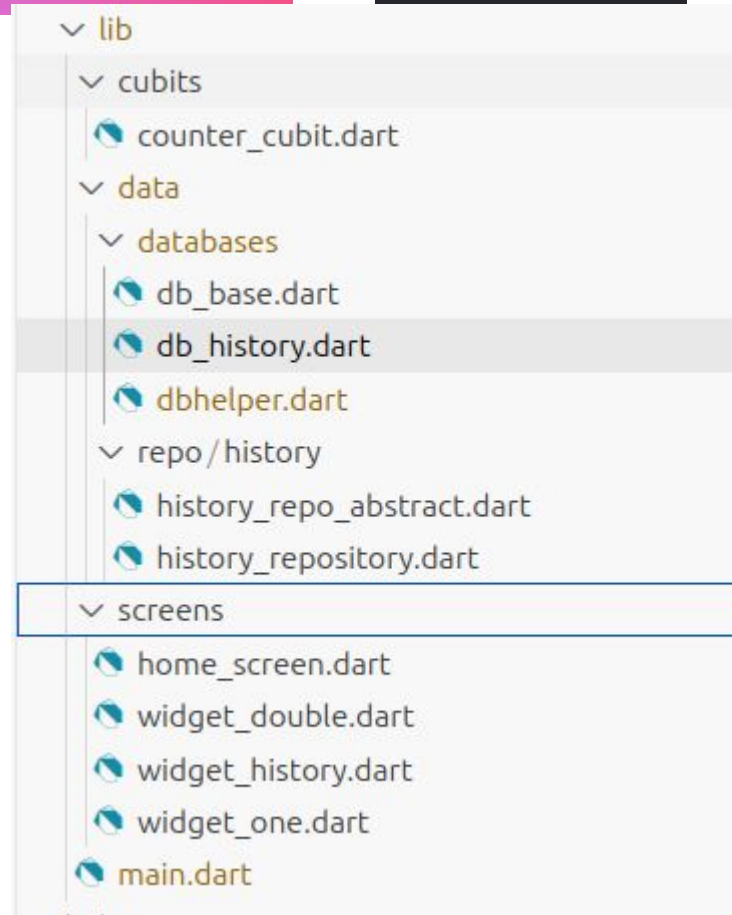
Screens / Widget ?

We don't change them...



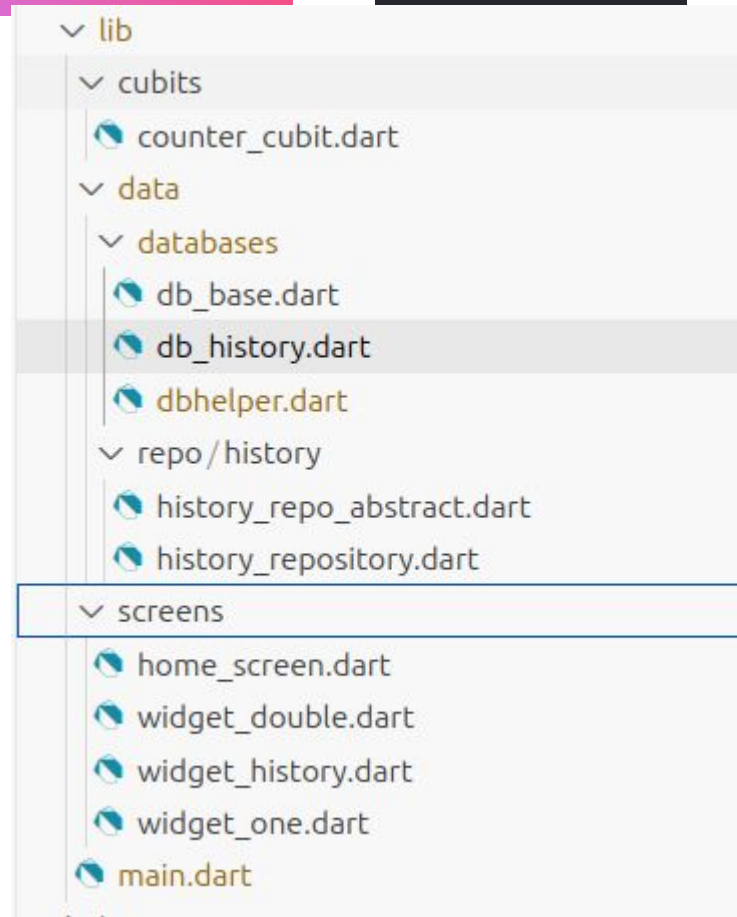
dbhelper.dart

```
class DBHelper {  
  static const database name = "IncrementHistoryV1.db";  
  static const _database_version = 1;  
  static var database;  
  
  static List<String> sql codes = [DBHistoryTable.sql_code];  
  static Future<Database> getDatabase() async {  
  
    if (database != null) {  
      return database;  
    }  
  
    database = openDatabase(  
      join(await getDatabasesPath(), _database_name),  
      onCreate: (database, version) {  
        sql codes.forEach((item) {  
          database.execute(item);  
        });  
      },  
      version: _database_version,  
      onUpgrade: (db, oldVersion, newVersion) {  
        //do nothing...  
      },  
    );  
    return database;  
  }  
}
```



db_base.dart

```
class DBBaseTable {  
  var db_table = 'TABLE_NAME_MUST_OVERRIDE';  
  
  Future<bool> insertRecord(Map<String, dynamic> data) async {  
    try {  
      final database = await DBHelper.getDatabase();  
      database.insert(  
        db_table,  
        data,  
        conflictAlgorithm: ConflictAlgorithm.replace,  
      );  
      return true;  
    } catch (e, stacktrace) { print('$e --> $stacktrace'); }  
    return false;  
  }  
  
  Future<List<Map>> getRecords() async {  
    try {  
      final database = await DBHelper.getDatabase();  
      var data = await database.rawQuery(  
        "select * from $db_table order by id DESC",  
      );  
      return data;  
    } catch (e, stacktrace) { print('$e --> $stacktrace'); }  
    return [];  
  }  
}
```

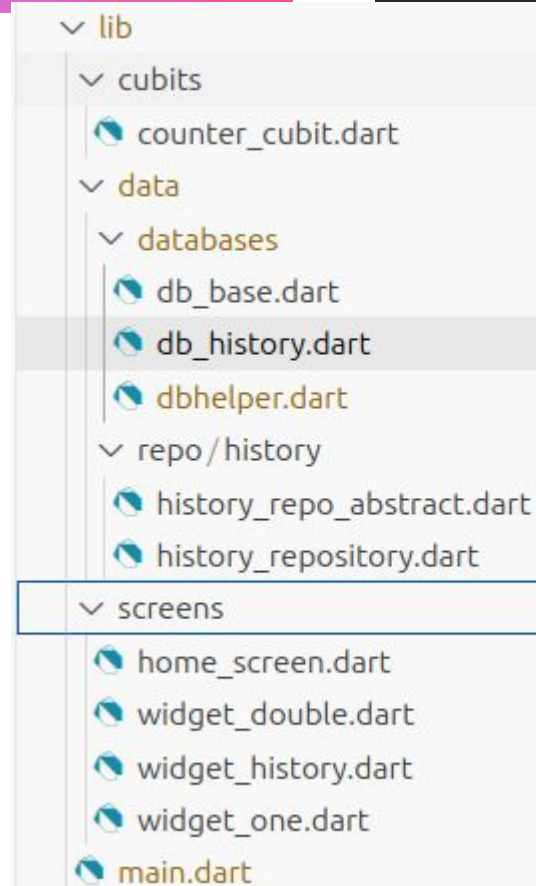


db_history.dart

```
import 'package:sqflite/sqflite.dart';

import 'db_base.dart';

class DBHistoryTable extends DBBaseTable {
  var db_table = 'history';
  static String sql_code = '''
    CREATE TABLE history (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      message TEXT,
      create_date TEXT
    )
  ''';
}
```



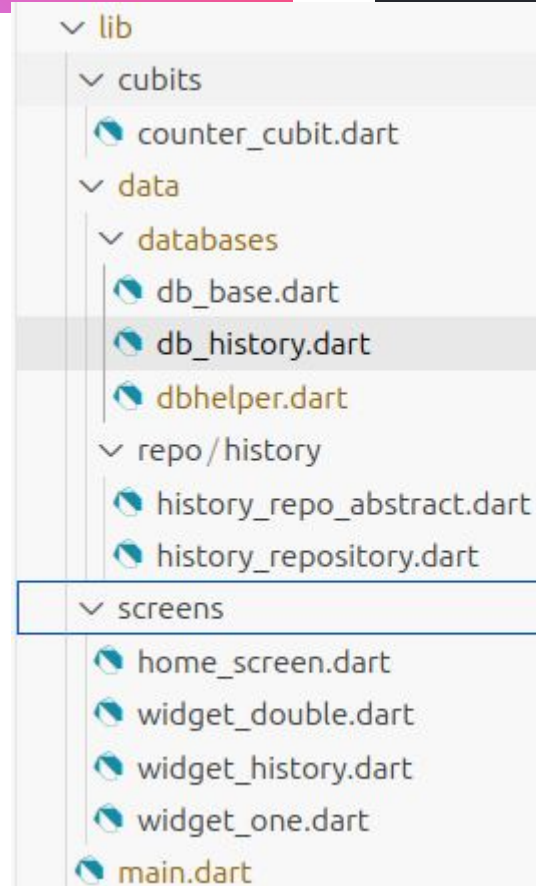
history_repo_abstract.dart

```
import 'history_repository.dart';

abstract class HistoryRepositoryBase {
  Future<List<String>> getData();
  Future<bool> insertData(String value);

  static HistoryRepositoryBase? _historyInstance;

  static HistoryRepositoryBase getInstance() {
    _historyInstance ??= HistoryRepository();
    return _historyInstance!; // For backend data
  }
}
```



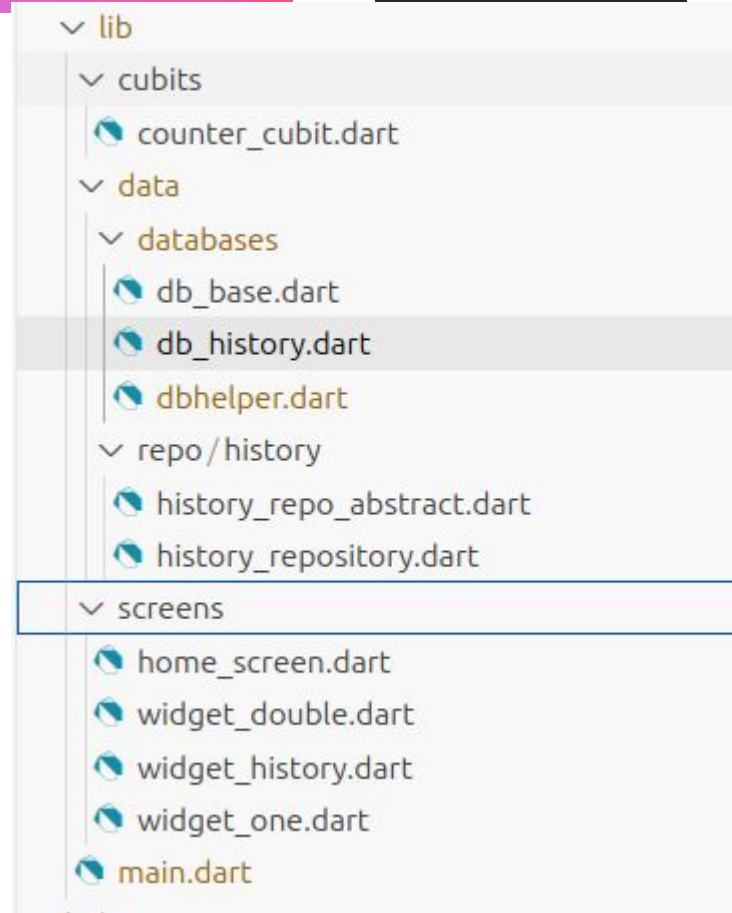
history_repository.dart

```
import '../.../databases/db_history.dart';
import 'history_repo_abstract.dart';

class HistoryRepository extends HistoryRepositoryBase {
  final DB_HISTORY = DBHistoryTable();

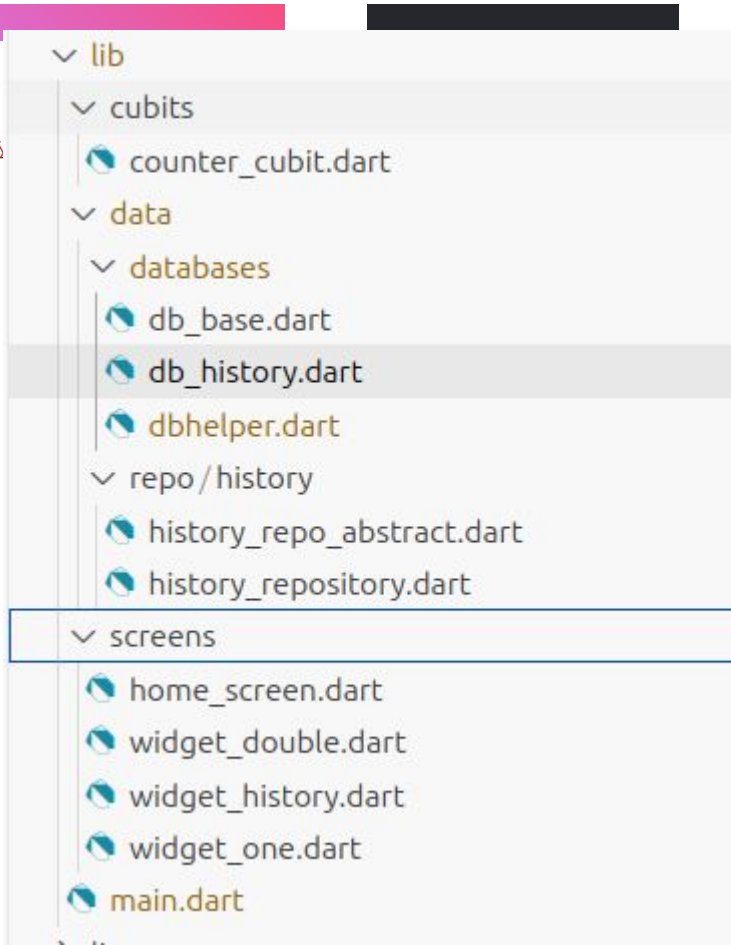
  @override
  Future<List<String>> getData() async {
    List<Map> obj = await DB_HISTORY.getRecords();
    List<String> result = [];
    obj.forEach((item) {
      result.add(item['message'] + ' ' + item['create_date']);
    });
    return result;
  }

  @override
  Future<bool> insertData(String item) async {
    DB_HISTORY.insertRecord({
      'message': item,
      'create_date': DateTime.now().toString(),
    });
    return true;
  }
}
```



counter_cubit.dart

```
class CounterCubit extends Cubit<Map<String, dynamic>> {  
  final repo = HistoryRepositoryBase.getInstance();  
  CounterCubit() : super({'status': 'loading', 'count': 0, 'history':  
    loadData();  
  }  
  Future<bool> loadData({int increment = 1}) async {  
    Map<String, dynamic> data = {'status': 'loading',  
      'count': increment,  
      'history': state['history'],  
    };  
    emit(data);  
    await Future.delayed(Duration(seconds: 3));  
    try {  
      final records = await repo.getData();  
      Map<String, dynamic> data2 = {  
        'status': 'done', 'count': increment, 'history': records};  
      emit(data2);  
    } catch (e, stack) {print('$e --> $stack');}  
    return true;  
  }  
  
  Future<bool> increment() async {  
    await repo.insertData('Incremented');  
    loadData(increment: state['count'] + 1);  
    return true;  
  }  
}
```

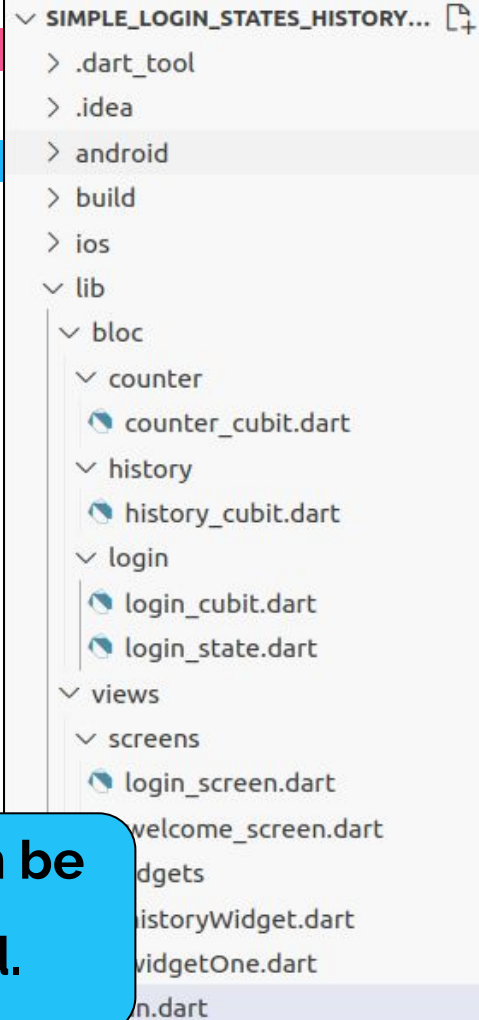


- lib
 - cubits
 - counter_cubit.dart
 - data
 - databases
 - db_base.dart
 - db_history.dart
 - dbhelper.dart
 - repo/history
 - history_repo_abstract.dart
 - history_repository.dart
 - screens
 - home_screen.dart
 - widget_double.dart
 - widget_history.dart
 - widget_one.dart
 - main.dart

main.dart

```
void main() {  
  runApp(const MainApp());  
}  
  
class MainApp extends StatelessWidget {  
  const MainApp({super.key});  
  
  @override  
  Widget build(BuildContext context) {  
    return MultiBlocProvider(  
      providers: [  
        BlocProvider(create: (_) => LoginCubit()),  
        BlocProvider(create: (_) => CounterCubit()),  
        BlocProvider(create: (_) => HistoryCubit()),  
      ],  
      child: MaterialApp(  
        home: LoginScreen(),  
        debugShowCheckedModeBanner: false,  
      ));  
  }  
}
```

I am providing the HistoryCubit Globally ? but it can be defined locally instead next to where it is needed.





Students' Questions

- Does it mean , we won't need to use at all `setState` ?

Asked by Heni



Students' Questions

- Does it mean , we won't need to use at all `setState` ?

Asked by Heni